

Discussion-6 MySQL- difference between stored procedure and functions

Discussion Topic:

Discuss the difference between stored procedure and functions.

Give an example of when you would use one instead of the other.

If you were a new database administrator, would it be easier to develop stored procedures or functions?

My Post:

Hello Class,

MySQL supports stored routines (procedures and functions) (MySQL, n.d.). Stored routines are a set of SQL statements that are compiled and stored on the server database, allowing them to be accessed by users. This article provides an overview of MySQL stored procedures and functions, and compares and evaluates their characteristics, differences, and use cases.

Store Procedures

Stored procedures can be defined as a set of SQL statements that can be invoked with the CALL keyword (Musgrave, 2024). Applications that have access to the database can CALL procedures stored on the database server. In other words, a stored procedure is a database object containing procedural SQL code that can be invoked at any time by an application or user to fetch and/or manipulate data. (Murach, 2019).

Syntax and creation of stored procedures:

```
CREATE PROCEDURE procedure_name (  
    [parameter_name_1 data_type]  
    [, parameter_name_2 data_type]...  
)  
-- sql_block -- This represents the body of the procedure
```

Example of creating/calling/dropping a procedure:

The example provided is from "Chapter 15: How to create stored procedures and functions. Murach's MySQL (3rd ed.)" By Murrah (2019). In this example, the procedure updates the credit_total column of the invoices table.

```
-- Change the standard delimiter from semicolon (;) to double slash (//)  
-- Semicolons are used within the procedure body,  
-- and we need a way to tell MySQL where the entire CREATE PROCEDURE statement ends,  
-- therefore, we need to define a delimiter for the procedure itself  
DELIMITER //  
  
-- Create a new stored procedure named 'update_invoices_credit_total'.  
-- This procedure takes two input parameters.  
CREATE PROCEDURE update_invoices_credit_total (  
    invoice_id_param INT,  
    credit_total_param DECIMAL(9,2)  
)  
-- Start of the procedure's body  
BEGIN  
    -- Declare a local variable to track SQL errors.  
    -- It's a TINYINT (often used for boolean flags) and defaults to FALSE (0).
```

```

DECLARE sql_error TINYINT DEFAULT FALSE;
SET sql_error = TRUE;

-- Start a new transaction. This allows us to group SQL statements together
START TRANSACTION;

-- MySQL/SQL statements
UPDATE invoices
SET credit_total = credit_total_param
WHERE invoice_id = invoice_id_param;

-- Check if an SQL error occurred during the UPDATE statement
IF sql_error = FALSE THEN
    COMMIT;
ELSE
    ROLLBACK;
END IF;
-- End of the procedure's body.
END //
-- Reset the delimiter back to the standard semicolon (;).
DELIMITER ;

```

Invoking Procedure:

```
CALL update_invoices_credit_total(56, 504);
```

Dropping the procedure

```
DROP PROCEDURE IF EXISTS update_invoices_credit_total;
```

Store Functions

Stored functions can be defined as a block of SQL code that can perform computations or transformations, and return a single scalar value (200ok Solutions 2024).

MySQL provides its own set of stored functions referred to as built-in functions, such as CONCAT(), NOW(), or SUM(). For a list of all the MySQL built-in functions, see: [MySQL Functions](#) by W³ School (n.d.). Users can also create their own stored functions, which are referred to as User-defined functions (UDFs).

Syntax and creation of stored functions:

```

CREATE FUNCTION function_name (
    [parameter_name_1 data_type]
    [, parameter_name_2 data_type]...
)
RETURNS data_type -- Specifies the data type of the value the function will return
-- Indicates if the function always produces the same result for the same input parameters
[NOT] DETERMINISTIC
-- Specifies the nature of SQL usage within the function
{CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA}
sql_block -- This represents the body of the function

```

Example of creating/calling/dropping a UDF function:

The example provided is from “Chapter 15: How to create stored procedures and functions. Murach’s MySQL (3rd ed.)” By Murrah (2019). In this example, the function returns the vendor ID that matches a vendor’s name.

```
-- Change the standard delimiter from semicolon (;) to double slash (//).
```

```

-- This is necessary because semicolons are used within the function body,
-- and we need a way to tell MySQL where the entire CREATE FUNCTION statement ends.
DELIMITER //

-- Create a new stored function named 'get_vendor_id'.
CREATE FUNCTION get_vendor_id (
    vendor_name_param VARCHAR(50)
)
-- return an INT (integer) value.
RETURNS INT
-- DETERMINISTIC: Indicates that this function will always return the same result
-- for the same input 'vendor_name_param', if data in the 'vendors' table doesn't change.
DETERMINISTIC
-- Indicates that the function contains SQL statements that read data
READS SQL DATA
-- start of the function body
BEGIN
    DECLARE vendor_id_var INT;

    SELECT vendor_id
    INTO vendor_id_var
    FROM vendors
    WHERE vendor_name = vendor_name_param;

    -- Return the value stored in 'vendor_id_var'.
    RETURN(vendor_id_var);
-- end of the function's body
END //
-- Reset the delimiter back to the standard semicolon (;).
DELIMITER ;

```

Invoking Function:

This was done in the same way MySQL functions are called within MySQL statements, often as part of an expression, for example, in a WHERE clause like:

```

SELECT invoice_number, invoice_total
FROM invoices
WHERE vendor_id = get_vendor_id('IBM'); -- Invoke/call/uses the function

```

Dropping the procedure

```

DROP FUNCTION IF EXISTS get_vendor_id;

```

Stored Procedures vs. Functions

The two stored routines are similar, but they differ in key aspects such as how they are called, what they can return, and how they interact with data. The table below provides a side-by-side comparison of their main differences.

Table 1

Stored Procedures and Functions Differences

	Stored Procedure	Function
Invocation	Can be called from an application that has access to the database using the <code>CALL</code> statement.	Can be called from within a SQL statement, like MySQL's built-in functions.

Return Values	Do not return a direct value like functions do. However, they can use <code>OUT</code> or <code>INOUT</code> parameters to pass values to the calling program. They can also pass result sets to the calling program using <code>SELECT</code> statements.	Are designed to always return a single value (a single piece of data, not a result set). The data type of the return value is specified and needs to be defined using the <code>RETURNS</code> keyword.
Data Modification	Can be used to modify data within a database, for example, by executing <code>INSERT</code> , <code>UPDATE</code> , or <code>DELETE</code> statements. They can manage transactions, including <code>COMMIT</code> and <code>ROLLBACK</code> operations.	It is technically possible to use functions to modify data when marked with the <code>MODIFIES SQL DATA</code> characteristic; however, functions are more commonly used to store procedures for data modification. Functions are generally used to read data or perform calculations, not to modify data.
Parameters	Can utilize <code>IN</code> (input), <code>OUT</code> (output), and <code>INOUT</code> (input/output) parameters.	Primarily use input parameters. Output parameters are technically possible, but they rarely make sense for the typical use case of a function.
Creation Syntax	Created using the <code>CREATE PROCEDURE</code> statement.	Created using the <code>CREATE FUNCTION</code> statement, which includes a <code>RETURNS</code> clause to define the data type of the output and can include characteristics like <code>DETERMINISTIC</code> or <code>READS SQL DATA</code> .

Note: The table provides a side-by-side comparison of the main differences between stored procedures and functions in MySQL.

Another aspect to consider is when to use one instead of the other; the choice depends heavily on the specific task's purpose. The table below provides a side-by-side comparison of the common scenarios or use cases for each routine.

Table 2
When to Choose a Stored Procedure vs. a Stored UDF Function

Choose Stored Procedure When:	Choose Stored Function When:
The task involves more than just returning a single value.	The task involves specific, well-defined computations or transformations that need to return a single value.
The task requires multiple SQL statements, conditional logic (<code>IF</code> , <code>CASE</code>), loops, and several sequences of execution. When orchestrating multi-step logical transactions or business processes is needed	The task needs to perform calculations within multiple queries, and when these calculations need to be reused.
The task's primary goal is to modify data, for example, when using statements such as <code>INSERT</code> , <code>UPDATE</code> , or <code>DELETE</code> data in one or more tables.	The task consists of custom date formatting, string manipulation.
The task consists of atomic operations (all changes succeed or all fail together), it requires <code>START TRANSACTION</code> , <code>COMMIT</code> , and <code>ROLLBACK</code> statements.	The task uses conditional logic that returns a boolean or comparable value.

The task needs to return several distinct pieces of data (using <code>OUT</code> or <code>INOUT</code> parameters) or one or more entire tables of data.	The task results will be used as part of a larger SQL statement, for example, within <code>SELECT</code> list, <code>WHERE</code> clause, <code>ORDER BY</code> , <code>INSERT/UPDATE</code> .
The task consists of DDL and DML statements for database maintenance tasks (archiving data, rebuilding indexes).	The task consists of read-only and is guaranteed not to modify any database data.

Note: The table provides a side-by-side comparison of common scenarios or use cases for stored procedure vs stored UDF function.

Below is an example of creating and using a simple procedure that gets all products belonging to a specific category, from the `copy_my_guitar_shop` schema (database). In this example, using a stored procedure rather than a stored function is better because the task involves more than just returning a single value; it returns to an application, for example, an entire result set (a table of data) consisting of multiple rows and columns. On the other hand, stored functions cannot return data tables, they can only return a single scalar value, making them unsuitable for this kind of task.

Figure 1
Example of a Simple Stored Procedure

```

1  USE copy_my_guitar_shop; -- Database to use
2
3  DELIMITER //
4
5  CREATE PROCEDURE get_products_by_category (
6      IN category_name_param VARCHAR(255) -- Input: the name of the category
7  )
8  BEGIN
9      SELECT
10         p.product_name,
11         p.product_code,
12         p.list_price,
13         p.description
14     FROM
15         products p
16     JOIN
17         categories c ON p.category_id = c.category_id
18     WHERE
19         c.category_name = category_name_param;
20 END //
21
22 DELIMITER ;
23
24 CALL get_products_by_category('Guitars');
25

```

product_name	product_code	list_price	description
Fender Stratocaster	strat	699.00	The Fender Stratocaster is the electric guitar de...
Gibson Les Paul	les_paul	1199.00	This Les Paul guitar offers a carved top and hu...
Gibson SG	sg	2517.00	This Gibson SG electric guitar takes the best of t...
Yamaha FG700S	fg700s	489.99	The Yamaha FG700S solid top acoustic guitar ha...
Washburn D10S	washburn	299.00	The Washburn D10S acoustic guitar is superbly ...
Rodriguez Caballero 11	rodriguez	415.00	Featuring a carefully chosen, solid Canadian ce...

Note: The figure illustrates the MySQL code to create and use a simple stored Procedure that gets product information based on the given category.

Below is an example of creating and using a simple UDF function that calculates the final price of an order item after applying its discount, from the `copy_my_guitar_shop` schema (database).

In this example, using a stored function rather than a stored procedure is better suited as the task performs a computation (calculating the final price) that returns a single value that can be used by the SELECT query, see code below. On the other hand, a stored procedure would be overkill for this task, as its underlying design introduces unnecessary overhead for this particular computation when compared to a stored function.

Figure 2

Example of a Simple UDF Function

The screenshot shows a MySQL IDE interface. On the left, the 'Navigator' pane displays the database schema 'copy_my_guitar_shop', with 'Functions' expanded to show 'f() calculate_final_item_price'. The main editor window, titled 'SQL File 1*', contains the following SQL code:

```

1  USE copy_my_guitar_shop; -- Database to use
2
3  DELIMITER //
4
5  CREATE FUNCTION calculate_final_item_price (
6      item_price_param DECIMAL(10,2), -- the original price of the item
7      discount_amount_param DECIMAL(10,2) -- the discount amount for the item
8  )
9      RETURNS DECIMAL(10,2)
10     DETERMINISTIC -- the function returns the same result for the same inputs
11     READS SQL DATA -- the function contains SQL statements that read data
12     BEGIN
13         DECLARE final_price DECIMAL(10,2);
14
15         -- Calculate the final price
16         SET final_price = item_price_param - discount_amount_param;
17
18         -- Ensure final price is not negative
19         IF final_price < 0 THEN
20             SET final_price = 0;
21         END IF;
22
23         RETURN final_price;
24     END //
25
26 DELIMITER ;
27
28 -- Function called from within a SQL statement
29 SELECT
30     item_id,
31     product_id,
32     item_price,
33     discount_amount,
34     -- calls the function within a query statement
35     calculate_final_item_price(item_price, discount_amount) AS final_item_price
36 FROM
37     order_items;
38
39

```

At the bottom, the 'Result Grid' shows the output of the query, displaying 12 rows of data with columns: item_id, product_id, item_price, discount_amount, and final_item_price.

	item_id	product_id	item_price	discount_amount	final_item_price
1	1	2	1199.00	359.70	839.30
2	2	4	489.99	186.20	303.79
3	3	3	2517.00	1308.84	1208.16
4	4	6	415.00	161.85	253.15
5	5	2	1199.00	359.70	839.30
6	6	5	299.00	0.00	299.00
7	7	5	299.00	0.00	299.00
8	8	1	699.00	209.70	489.30
9	9	7	799.99	240.00	559.99
10	10	9	699.99	210.00	489.99
11	11	10	799.99	120.00	679.99
12	12	1	699.00	209.70	489.30

Note: The figure illustrates the MySQL code to create and use a simple stored UDF function that calculates the final price of an order item after applying its discount based on the given category.

Considering the routine differences and complexities as shown in Tables 1 and 2, and in Figures 1 and 2. New database administrators may find stored UDF functions easier to develop. Because they return a single value and have limited data modification capabilities, their development can be more straightforward/simpler, and they are safer to use than stored procedures. In other words, their limited scope and syntax can feel familiar to developers with programming experience, and their limited scope and data modification capacity make them safer to use.

Stored procedures, on the other hand, can quickly become extremely complex as they have a wider scope than stored functions. They can incorporate control flow, error handling, transaction management, and data modifications. Consequently, for a new database administrator, stored procedures may be more challenging and have a steeper learning curve compared to stored UDF functions.

As a new database administrator is best to start with functions and then gradually progress from procedures performing basic operations to procedures with more complex logic.

To summarize, stored procedures can be defined as a set of SQL statements that can be invoked using the CALL keyword. They are database objects containing procedural SQL code that can be invoked at any time by an application or user to fetch and/or manipulate data. On the other hand, a stored function is a block of SQL code that can perform computations or transformations and return a single scalar value. MySQL provides its own set of stored functions, also called built-in functions, such as CONCAT(), NOW(), or SUM(). Users can create their own stored functions, called User-defined functions (UDFs). Although both stored procedures and user-defined functions provide code modularity, improved performance, and encapsulate business logic, their use cases differ. Stored procedures are used for complex data manipulation, transaction control, or when operations need to return multiple values or result sets; on the other hand, stored functions are used for calculations and transformations that return a single scalar value. Therefore, selecting the right routine for a specific task is critical for the performance and functionality of relational databases.

-Alex

References:

200ok Solutions (2024, September 20). MySQL stored procedures vs. functions: When and how to use them. 200ok Solutions. <https://200oksolutions.com/blog/mysql-stored-procedures-vs-functions/>

Murach, J. (2019). Chapter 15: How to create stored procedures and functions. *Murach's MySQL (3rd ed.)*. Murach Books. ISBN: 9781943872367

Musgrave, Z. (2024, January 17). *MySQL stored procedures: How and why with examples*. DoltHub Blog. <https://www.dolthub.com/blog/2024-01-17-writing-mysql-procedures/>

MySQL (n.d.). 27.2 Using stored routines. *MySQL 8.4 reference manual*. <https://dev.mysql.com/doc/refman/8.4/en/stored-routines.html>

W³ School (n.d.). MySQL functions. *SQL reference*. W³ School. https://www.w3schools.com/SQL/sql_ref_mysql.asp